الجامعة الدولية للرباط
ⵜⴰⵙⴷⴰⵡⵉⵜ ⵜⴰⵅⵔⴰⵟⵉⵜ ⵏ ⵔⵕⴱⴰⵟ
Université Internationale de Rabat
L'UNIVERSITÉ INNOVANTE

Ecole Supérieure
d'Informatique et du Numérique
COLLEGE OF ENGINEERING & ARCHITECTURE



Cloud Foundations



Chapter 7: Serverless Functions

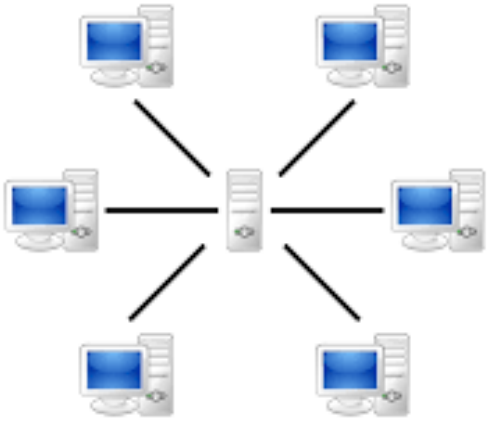
PR. OUASSIM KARRAKCHOU

UNIVERSITE INTERNATIONALE DE RABAT

Academic year: 2026-2027

Serverless Computing

The Traditional Client-Server Architecture

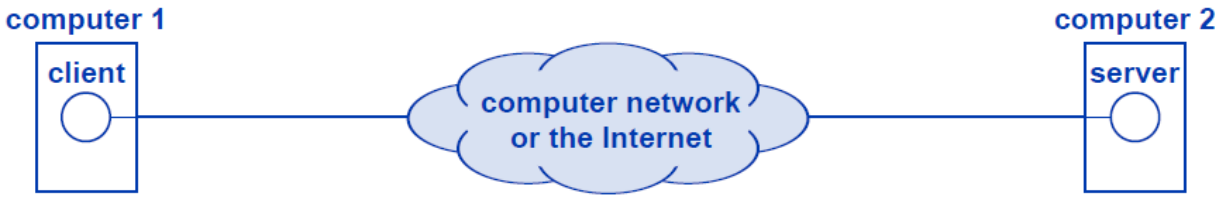


To understand the advantages of serverless computing, one must first understand the traditional client-server architecture that network applications use.

Based on your knowledge, how does the client-server architecture work?

Serverless Computing

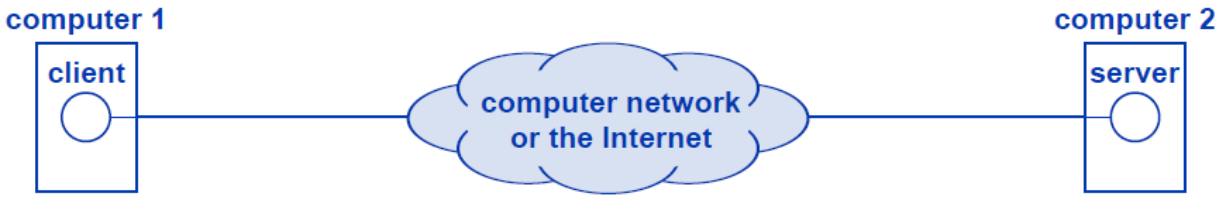
The Traditional Client-Server Architecture



- When application programs communicate over a network, they follow the client-server paradigm which divides applications into two categories based on how they initiate communication.
- One of the two programs must be designed to start first and remain ready to be contacted. It is the server.
- The other application must be designed to initiate contact. It is the client.

Serverless Computing

Scaling the Traditional Client-Server Architecture

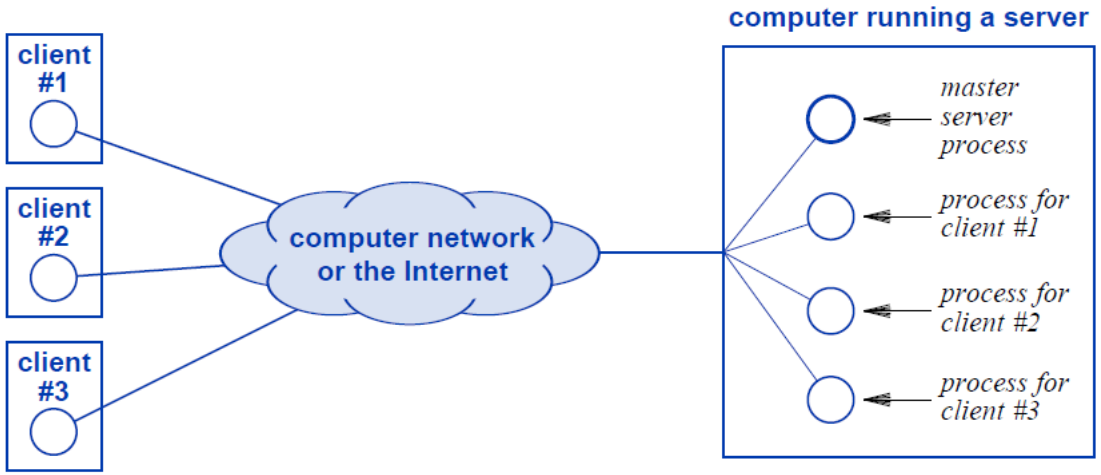


- As we saw in previous chapters, one of the main issues that cloud technologies aim to solve is the issue of scalability.
- Do you know how traditional servers scale to handle concurrent requests?
- In a traditional client-server architecture, a server scales by using concurrent execution to handle multiple clients at the same time.
- That is, the server does not handle one client and then go on to the next. Instead, the server repeatedly waits for a client to initiate communication and then creates a concurrent process to handle the client.



Serverless Computing

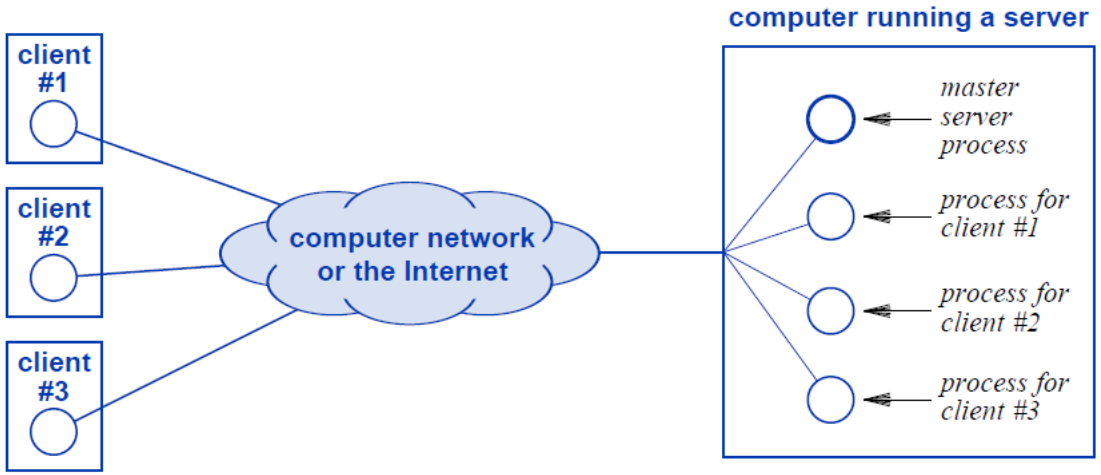
Scaling the Traditional Client-Server Architecture



- At any time, N +1 server processes are running:
  - A master server that clients contact
  - N instances of the server that are each handling an active client

Serverless Computing

Scaling the Traditional Client-Server Architecture



- The figure makes it appear that four independent application programs are running on the server computer. In fact, they are all copies of one server application.

- When a connection arrives from a new client, the master server copy calls fork to create a new process to handle the client.



Serverless Computing

Scaling the Traditional Client-Server Architecture

- Conceptually, the application for a concurrent server integrates two aspects of a server into a single application program:
    - Fulfilling the service being accessed: A client contacts a server to access a service. Typically, a client sends a request to which the server sends a reply. The main function of a server lies in interacting with clients to fulfill clients' requests.
    - Replicating and scaling: The second aspect of a server arises from the techniques used to replicate and scale the server. A traditional concurrent server uses contact from a new client to trigger the creation of a separate process to handle the client.



Serverless Computing

Scaling a Server in a Cloud Environment

- Now that you saw how a traditional server is scaled, what do you think are the problems that arise when using this scaling method in a cloud environment?
- A traditional concurrent server replicates copies of itself automatically to handle multiple clients simultaneously.
- However, all copies must run on the same physical computer.
- The approach does not work well in a cloud environment because cloud systems achieve large scale by replicating instances across many physical machines.
- Consequently, complex software systems must be used to run servers in a cloud data center.



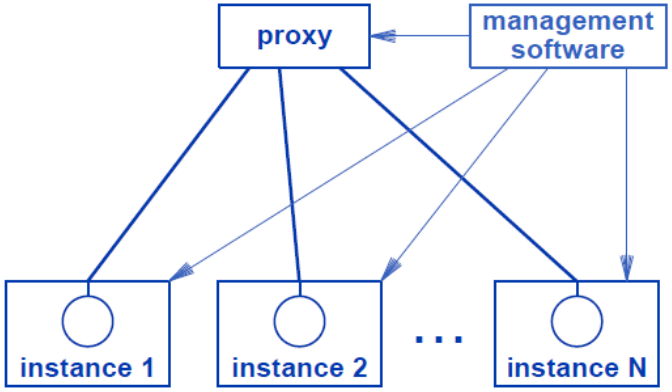
Serverless Computing

Scaling a Server in a Cloud Environment

- There are globally two main systems needed to scale a server in a cloud environment:
  - Instance management: deploy copies of the server as needed.
  - Network communication handling: automatically forward traffic to the correct instance.
- A large set of management technologies exist that can be used to deploy and operate services, including orchestration systems, proxies, load balancers, and service mesh management software..
- In terms of cost, many of the software technologies follow the open-source model, making them free.

Serverless Computing

Scaling a Server in a Cloud Environment



- It seems that open-source software allows a cloud customer to port existing servers to the cloud and then deploy and scale the servers at little extra cost.

- That is, a customer only needs to have basic server software, lease a set of VMs, and use open-source software to handle deployment and scaling.

Do you agree with the statement above regarding the cost?



Serverless Computing

Cost of Scaling a Server in a Cloud Environment

- Unfortunately, two costs can be significant:
  - Unused Capacity: Because setting up a VM takes time, a customer must plan ahead by allocating sufficient VMs to handle the expected peak load. As a result, the customer must pay for VMs that remain idle during off-peak times.
  - Expertise and Training: Although open-source management systems are free, using such technologies effectively and safely requires a significant amount of expertise that lies outside the typical expertise of a software engineer. Thus, a customer must hire experts. A customer that uses multi-cloud, must have expertise for each cloud system. Furthermore, because cloud technologies continue to evolve, the customer must pay for training to keep their staff up to date.



Serverless Computing

The Serverless computing paradigm

- In response to the need for server management, cloud providers created a way that a cloud customer can pay to have the provider handle all tasks associated with running the customer's servers.
- The industry has referred to the name Serverless computing to refer to the approach.
- To an outsider, the name seems inappropriate because the underlying system does indeed run servers and uses the client-server model of interaction.
- However, providers assert that from a customer's point of view, the facilities are "serverless" in the sense that the customer can build and run software to fulfill users' requests without thinking about the deployment and replication of servers, without configuring network names and addresses for servers, and without leasing VMs to run the servers.



Serverless Computing

The Serverless computing paradigm

- To help eliminate confusion, the industry sometimes uses the alternative name Function as a Service (FaaS).
- The name arises because a cloud customer writes code that performs the main function of the server and allows the provider to handle deployment.
- To summarize, the serverless computing approach allows a cloud customer to avoid dealing with servers by paying a cloud provider to deploy and scale the customer's servers.
- Because a provider amortizes the cost of expertise among all customers and only charges for the time a server is used, the serverless approach can cost a customer less than running servers themselves.



Serverless Computing

The Serverless computing paradigm

What happens if no one uses a particular server?

- In some cases, a provider imposes a minimum monthly fee.
- However, other providers offer service without a minimum fee, meaning that if no one uses a server in a given period, the customer pays nothing. Industry uses the term scale to zero to refer to those services.
- In terms of accommodating large numbers of clients, the serverless approach offers great flexibility.
- Unlike a self-managed set of servers that require a customer to plan for a peak load, serverless computing allows arbitrary scale. Industry uses the term scale to infinity to characterize a serverless system that does not place a limit on scale.



Serverless Computing

Serverless Architecture

Based on what you know, how is a serverless architecture implemented?

- The answer lies in combining technologies and architectures covered in previous chapters.
- To make it feasible to deploy servers quickly, serverless technologies use containers.
- To deploy and manage a server, serverless systems use orchestration.

So how is serverless different from an orchestrated containers architecture?



Serverless Computing

Serverless Architecture

- Despite building on existing technologies, the serverless approach introduces two key features that distinguish it from the traditional server approach:
  - Adherence to an event-driven paradigm
  - The use of stateless servers
- To explain these two features, we have to discuss the concept of event and state.
- The underlying cloud system generates events when changes occur (e.g., when a physical server fails). In addition, serverless systems count each server access (e.g., and HTTP request) as an event.
- In the serverless architecture, calls to the service are only triggered by the reception of an event.



Serverless Computing

Stateless Servers

- We use the term state to refer to data related to clients that a server stores internally.
- The term stateful server refers to a server that stores state information; and the term stateless server refers to a server that does not store state information.
- Stateful servers store information for two reasons. First, keeping information related to a given client allows a server to provide continuity across multiple contacts by the client. Second, keeping state information allows a server to share information among multiple clients.
- A stateful approach works well for a traditional server because the server runs on a single computer where the servers can use mechanisms like shared memory to access and update state information.



Serverless Computing

Stateless Servers

- The stateful approach does not work well for a server that runs in a data center and handles large scale.
- To achieve scale, orchestration systems deploy instances on multiple physical computers, making shared memory impossible.
- To handle microservices, containers are designed with a short lifetime: the container starts, performs one function, and exits. Thus, state information does not persist for more than one client connection.
- To capture the idea that serverless computing focuses on running a single, stateless function in each container (i.e., FaaS), some engineers say that serverless computing runs stateless functions.



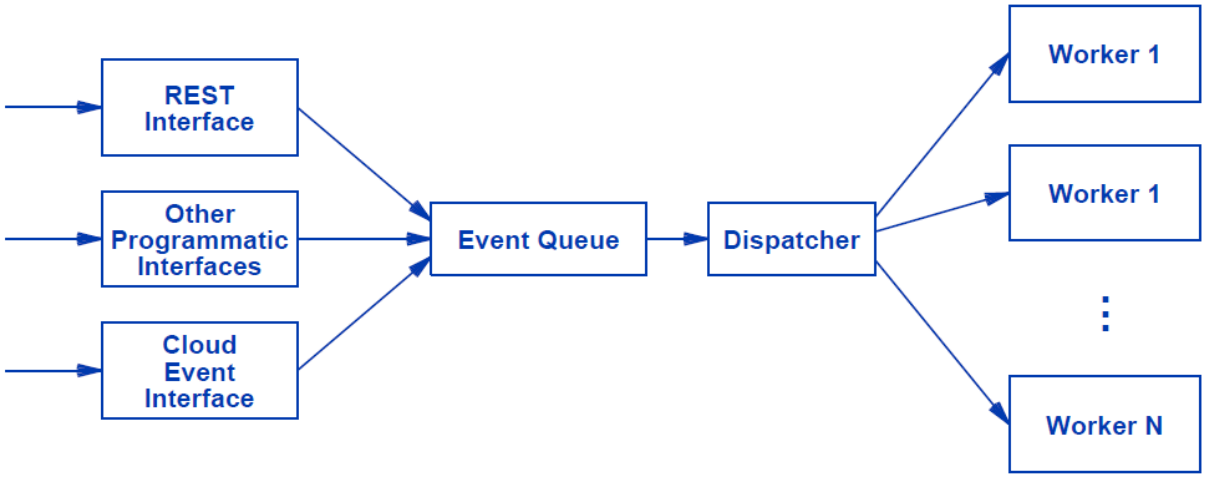
Serverless Computing

Stateless Servers

- It is important to distinguish between a stateful server and a server that keeps data in a database or persistent storage.
- Statefulness only refers to the information a server keeps in memory while the server runs. When a server exits, state information disappears.
- In contrast, data stored on an external storage system (e.g., a database, a file on NAS, or an object store) persists after the server exits.
- A server can use external storage to achieve the same effect as statefulness — the server saves a copy of internal data before exiting and reads back the data when it starts running.
- Stored data does not count as state information because the data is not lost when the server exits.

Serverless Computing

Serverless Infrastructure Architecture



- The main components include an event queue, a set of interface components that insert events into the queue, and a dispatcher that repeatedly extracts an event and assigns a worker node to process the event. Worker nodes run the server code.

- In addition to a REST interface that accepts contact via HTTP, some serverless systems provide a management interface, or other programmatic interfaces to interact with the system.



Containers

Summary

- *Serverless computing follows the traditional client-server paradigm in which one or more clients initiate contact with a server. Unlike a traditional concurrent server that is limited to one computer, serverless computing uses cloud technologies that allow it to scale arbitrarily.*

- *The main motivation for serverless computing lies in its economic benefits: customers do not need to maintain staff with expertise and do not pay for idle VMs.*

- *Serverless systems use an event- based paradigm in which each change in the cloud system and each contact from a client becomes an event that is added to a queue. A dispatcher repeatedly extracts events and assigns them to stateless worker nodes to handle.*